

VL-11: Primitive Rekursion

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Wiederholung

Wdh.: LOOP versus WHILE

Die Ackermann Funktion $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ ist folgendermassen definiert:

$$\begin{aligned} A(0, n) &= n + 1 && \text{für } n \geq 0 \\ A(m + 1, 0) &= A(m, 1) && \text{für } m \geq 0 \\ A(m + 1, n + 1) &= A(m, A(m + 1, n)) && \text{für } m, n \geq 0 \end{aligned}$$

Satz

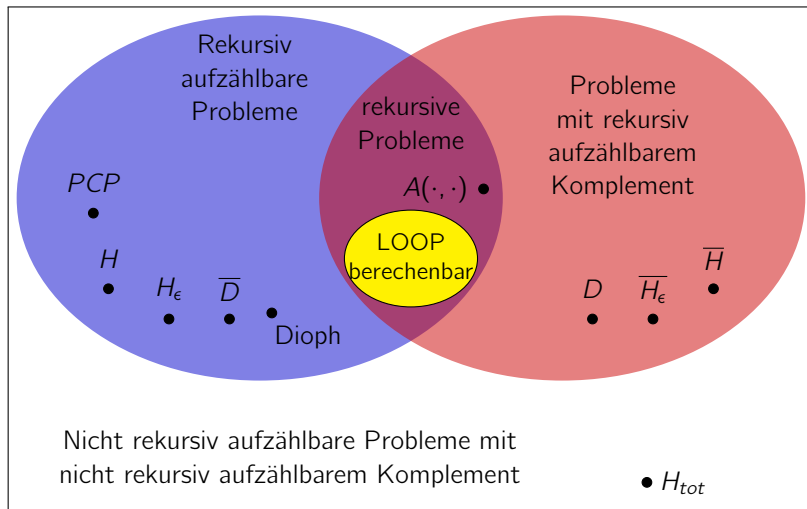
Die Ackermann Funktion ist nicht LOOP-berechenbar.

Da die Ackermann Funktion (durch eine TM) berechenbar ist, folgt:

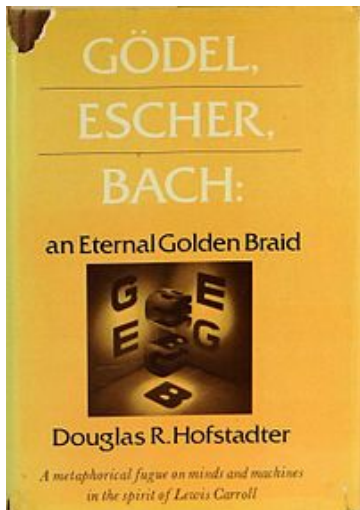
Satz

Die Menge der LOOP-berechenbaren Funktionen bildet eine **echte Teilmenge** der berechenbaren Funktionen.

Wdh.: Die Berechenbarkeitslandschaft



Douglas Hofstadter: "Gödel, Escher, Bach" (1979)



This book looks at the surprising points of contact between the music of **Bach**, the artwork of **Escher**, and the mathematics of **Gödel**. It also looks at the prospects for computers and artificial intelligence for mimicking human thought.

BlooP is a non-Turing-complete programming language in Hofstadter's book whose flow structure is a bounded loop. All programs in the language must terminate, and this language can only express **primitive recursive functions**.

Vorlesung VL-11

Primitiv rekursive Funktionen

- Primitiv rekursive Funktionen
- Äquivalenz zu LOOP
- μ -rekursive Funktionen
- Äquivalenz zu WHILE

Die primitiv rekursiven Funktionen bilden eine Unterfamilie der Funktionen $\mathbb{N}^k \rightarrow \mathbb{N}$ mit $k \geq 1$.

Sie sind induktiv definiert, und werden durch zwei **Operationen** aus den sogenannten **Basisfunktionen** zusammengebaut.

Die folgenden drei Funktionsmengen bilden die Basisfunktionen unter den primitiv rekursiven Funktionen:

- Alle **konstanten Funktionen** sind primitiv rekursiv.
- Alle identischen Abbildungen (**Projektionen** auf eine der Komponenten) sind primitiv rekursiv.
- Die **Nachfolgerfunktion** $\text{succ}(n) = n + 1$ ist primitiv rekursiv.

Anmerkung: Die Projektion $\mathbb{N}^k \rightarrow \mathbb{N}$ auf die p -te von k Komponenten wird manchmal mit $\pi_{k,p}$ bezeichnet.

Die beiden Operationen

Jede **Komposition** von primitiv rekursiven Funktionen ist primitiv rekursiv.

Wenn die Funktion $g : \mathbb{N}^k \rightarrow \mathbb{N}$ primitiv rekursiv ist und wenn die k Funktionen $h_1, \dots, h_k : \mathbb{N}^\ell \rightarrow \mathbb{N}$ primitiv rekursiv sind, so ist auch die folgende Funktion $f : \mathbb{N}^\ell \rightarrow \mathbb{N}$ primitiv rekursiv:

$$f(x_1, \dots, x_\ell) = g(h_1(x_1, \dots, x_\ell), \dots, h_k(x_1, \dots, x_\ell))$$

Jede Funktion, die durch **primitive Rekursion** aus primitiv rekursiven Funktionen entsteht ist primitiv rekursiv.

Wenn die beiden Funktionen $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ primitiv rekursiv sind, so ist auch $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ primitiv rekursiv:

$$\begin{aligned} f(0, x_1, \dots, x_k) &= g(x_1, \dots, x_k) \\ f(n+1, x_1, \dots, x_k) &= h(n, f(n, x_1, \dots, x_k), x_1, \dots, x_k) \end{aligned}$$

Intuition zu primitiv rekursiven Funktionen

Beobachtung: Jede primitiv rekursive Funktion ist berechenbar und total.

- Klar für Basisfunktionen. Und die Komposition von berechenbaren und totalen Funktionen ist ebenfalls berechenbar und total.

Intuition zu primitiv rekursiven Funktionen

Beobachtung: Jede primitiv rekursive Funktion ist berechenbar und total.

- Klar für Basisfunktionen. Und die Komposition von berechenbaren und totalen Funktionen ist ebenfalls berechenbar und total.
- Angenommen, die beiden Funktionen $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ sind berechenbar. Dann berechnen wir der Reihe nach:

$$y_0 := g(x_1, \dots, x_k) = f(0, x_1, \dots, x_k)$$

$$y_1 := h(0, y_0, x_1, \dots, x_k) = f(1, x_1, \dots, x_k)$$

$$y_2 := h(1, y_1, x_1, \dots, x_k) = f(2, x_1, \dots, x_k)$$

$$y_3 := h(2, y_2, x_1, \dots, x_k) = f(3, x_1, \dots, x_k)$$

$$\vdots \quad \vdots$$

$$y_n := h(n-1, y_{n-1}, x_1, \dots, x_k) = f(n, x_1, \dots, x_k)$$

Damit haben wir dann auch $f(n, x_1, \dots, x_k) = y_n$ berechnet.

Primitiv rekursive Beispiele

Beispiel: Addition

Die Additionsfunktion $\text{add} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{add}(x, y) = x + y$ ist primitiv rekursiv.

$$\text{add}(0, x) = x$$

$$\text{add}(n + 1, x) = \text{succ}(\text{add}(n, x))$$

Genauer: Die Additionsfunktion $\text{add} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der Projektion $g(x) = \pi_{1,1}(x) = x$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{succ}(\pi_{3,2}(a, b, c))$:

$$\text{add}(0, x) = g(x)$$

$$\text{add}(n + 1, x) = h(n, \text{add}(n, x), x)$$

Beispiel: Multiplikation

Die Multiplikationsfunktion $\text{mult} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{mult}(x, y) = x \cdot y$ ist primitiv rekursiv.

$$\text{mult}(0, x) = 0$$

$$\text{mult}(n + 1, x) = \text{add}(\text{mult}(n, x), x)$$

Genauer: Die Multiplikationsfunktion $\text{mult} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der konstanten Abbildung $g(x) = 0$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{add}(\pi_{3,2}(a, b, c), \pi_{3,3}(a, b, c))$:

$$\text{mult}(0, x) = g(x)$$

$$\text{mult}(n + 1, x) = h(n, \text{mult}(n, x), x)$$

Beispiel: Vorgänger

Die Vorgängerfunktion $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ mit $\text{pred}(x) = \max\{x - 1, 0\}$ ist primitiv rekursiv.

$$\text{pred}(0) = 0$$

$$\text{pred}(n + 1) = n$$

Genauer: Die Vorgängerfunktion $\text{pred} : \mathbb{N} \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus der konstanten Abbildung $g() = 0$ und aus der Projektion $h(a, b) = \pi_{2,1}(a, b) = a$:

$$\text{pred}(0) = g()$$

$$\text{pred}(n + 1) = h(n, \text{pred}(n))$$

Beispiel: Subtraktion

Die (modifizierte) Subtraktionsfunktion $\text{sub} : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $\text{sub}(x, y) = x \dot{-} y = \max\{x - y, 0\}$ ist primitiv rekursiv.

Da die zugrunde liegende primitive Rekursion über die erste Komponente läuft, definieren wir zunächst die Hilfsfunktion $\text{subAux}(y, x) = x \dot{-} y$.

$$\text{subAux}(0, x) = x$$

$$\text{subAux}(n + 1, x) = \text{pred}(\text{subAux}(n, x))$$

Genauer: Die Hilfsfunktion $\text{subAux} : \mathbb{N}^2 \rightarrow \mathbb{N}$ entsteht aus der Projektion $g(x) = \pi_{1,1}(x) = x$ und aus der primitiv rekursiven Funktion $h(a, b, c) = \text{pred}(\pi_{3,2}(a, b, c))$:

$$\text{subAux}(0, x) = g(x)$$

$$\text{subAux}(n + 1, x) = h(n, \text{subAux}(n, x), x)$$

Danach definieren wir $\text{sub}(x, y) = \text{subAux}(\pi_{2,2}(x, y), \pi_{2,1}(x, y))$.

Weitere primitiv rekursive Beispiele (1)

Notation: Für ein logisches **Prädikat** verwenden wir die Schreibweise $[\text{Prädikat}]$, wobei

$[\text{Prädikat}] = 0$ falls **Prädikat** den Wahrheitswert **falsch** hat, und

$[\text{Prädikat}] = 1$ falls **Prädikat** den Wahrheitswert **wahr** hat.

Übung

Die folgenden Funktionen sind primitiv rekursiv:

- Signum: $f : \mathbb{N} \rightarrow \{0, 1\}$ mit $f(x) = [x \geq 1]$
- Gleichheit: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x = y]$
- Kleiner: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x < y]$
- Grösser: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x > y]$
- Kleiner-gleich: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x \leq y]$
- Grösser-gleich: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x \geq y]$

Übung

Die folgenden Funktionen sind primitiv rekursiv:

- Maximum: $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $f(x, y) = \max\{x, y\}$
- Minimum: $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $f(x, y) = \min\{x, y\}$
- Division: $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $f(x, y) = \lfloor x/y \rfloor$
- Teilbarkeit: $f : \mathbb{N}^2 \rightarrow \{0, 1\}$ mit $f(x, y) = [x|y]$
- Parität: $f : \mathbb{N} \rightarrow \{0, 1\}$ mit $f(x) = [x \text{ gerade}]$
- Parität: $f : \mathbb{N} \rightarrow \{0, 1\}$ mit $f(x) = [x \text{ ungerade}]$
- Primzahl: $f : \mathbb{N} \rightarrow \{0, 1\}$ mit $f(x) = [x \text{ prim}]$

**Ein wichtiges Werkzeug:
Primitiv rekursive Bijektionen**

Eine Bijektion (1)

Die Funktion $\text{binom}_2 : \mathbb{N} \rightarrow \mathbb{N}$ mit

$$\text{binom}_2(x) = \binom{x}{2} = \frac{1}{2}x(x-1) = 0 + 1 + 2 + \cdots + (x-1)$$

ist primitiv rekursiv.

$$\text{binom}_2(0) = 0$$

$$\text{binom}_2(n+1) = \text{add}(n, \text{binom}_2(n))$$

Die Funktion $\beta : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit

$$\beta(x, y) = \binom{x+y+1}{2} + x$$

ist primitiv rekursiv.

Eine Bijektion (2)

Beobachtung

Die Funktion $\beta(x, y) = \binom{x+y+1}{2} + x$ ist eine Bijektion zwischen $\mathbb{N}^2$ und $\mathbb{N}$.

x/y	0	1	2	3	4	5	6	...
0	0	1	3	6	10	15	21	...
1	2	4	7	11	16	22	29	...
2	5	8	12	17	23	30	38	...
3	9	13	18	24	31	39	48	...
4	14	19	25	32	40	49	59	...
5	20	26	33	41	50	60	71	...
6	27	34	42	51	61	72	84	...
7	35	43	52	62	73	85	98	...

Die Umkehrfunktion der Bijektion (1)

Übung

Beweisen Sie: Für alle $x, y \in \mathbb{N}$ gilt $x \leq \beta(x, y)$ und $y \leq \beta(x, y)$.

Übung

Beweisen Sie: Für alle $x, y \in \mathbb{N}$ gilt $x \leq \beta(x, y)$ und $y \leq \beta(x, y)$.

- Als nächstes Ziel wollen wir nun für eine gegebene Zahl n die entsprechende (eindeutig festgelegte) Zahl $x \in \mathbb{N}$ in der Gleichung $\beta(x, y) = n$ bestimmen.
- Triviale Möglichkeit: Teste alle Paare $(x, y) \in \mathbb{N}^2$.
Klügere Möglichkeit: Teste alle Paare $(x, y) \in \{0, 1, \dots, n\}^2$.

Die Umkehrfunktion der Bijektion (2)

- Die Funktion $f : \mathbb{N}^3 \rightarrow \{0, 1\}$ mit $f(x, y, n) = [\beta(x, y) = n]$ ist primitiv rekursiv.

Die Umkehrfunktion der Bijektion (2)

- Die Funktion $f : \mathbb{N}^3 \rightarrow \{0, 1\}$ mit $f(x, y, n) = [\beta(x, y) = n]$ ist primitiv rekursiv.
- Die Funktion $g : \mathbb{N}^3 \rightarrow \{0, 1\}$ mit $g(x, k, n) = [\exists y \leq k : \beta(x, y) = n]$ ist primitiv rekursiv:

$$g(x, 0, n) = f(x, 0, n)$$

$$g(x, k, n) = [f(x, k, n) = 1 \vee g(x, k - 1, n) = 1]$$

Die Umkehrfunktion der Bijektion (2)

- Die Funktion $f : \mathbb{N}^3 \rightarrow \{0, 1\}$ mit $f(x, y, n) = [\beta(x, y) = n]$ ist primitiv rekursiv.
- Die Funktion $g : \mathbb{N}^3 \rightarrow \{0, 1\}$ mit $g(x, k, n) = [\exists y \leq k : \beta(x, y) = n]$ ist primitiv rekursiv:

$$g(x, 0, n) = f(x, 0, n)$$

$$g(x, k, n) = [f(x, k, n) = 1 \vee g(x, k - 1, n) = 1]$$

- Wir definieren $\max \emptyset = 0$. Die Funktion $h : \mathbb{N}^3 \rightarrow \mathbb{N}$ mit $h(m, k, n) = \max\{x \leq m : \exists y \leq k : \beta(x, y) = n\}$ ist primitiv rekursiv:

$$h(0, k, n) = 0$$

$$h(m, k, n) = \max\{h(m - 1, k, n), m \cdot g(m, k, n)\}$$

Die Umkehrfunktion der Bijektion (3)

Daraus folgt sofort:

Die Funktion $\gamma : \mathbb{N} \rightarrow \mathbb{N}$ mit $\gamma(n) = h(n, n, n)$ ist primitiv rekursiv

- Also: $\gamma(n) = h(n, n, n)$ ist die grösste Zahl $x \leq n$, für die eine Zahl $y \leq n$ existiert, sodass $\beta(x, y) = n$ gilt.

Die Umkehrfunktion der Bijektion (3)

Daraus folgt sofort:

Die Funktion $\gamma : \mathbb{N} \rightarrow \mathbb{N}$ mit $\gamma(n) = h(n, n, n)$ ist primitiv rekursiv

- Also: $\gamma(n) = h(n, n, n)$ ist die grösste Zahl $x \leq n$, für die eine Zahl $y \leq n$ existiert, sodass $\beta(x, y) = n$ gilt.
- Für jedes $n \in \mathbb{N}$ gibt es ein einziges Zahlenpaar $(x, y) \in \mathbb{N}^2$ mit $\beta(x, y) = n$. Die Funktion $\gamma(n)$ gibt uns daher die (eindeutig bestimmte) Zahl x in dieser Gleichung an.

Die Umkehrfunktion der Bijektion (3)

Daraus folgt sofort:

Die Funktion $\gamma : \mathbb{N} \rightarrow \mathbb{N}$ mit $\gamma(n) = h(n, n, n)$ ist primitiv rekursiv

- Also: $\gamma(n) = h(n, n, n)$ ist die grösste Zahl $x \leq n$, für die eine Zahl $y \leq n$ existiert, sodass $\beta(x, y) = n$ gilt.
- Für jedes $n \in \mathbb{N}$ gibt es ein einziges Zahlenpaar $(x, y) \in \mathbb{N}^2$ mit $\beta(x, y) = n$. Die Funktion $\gamma(n)$ gibt uns daher die (eindeutig bestimmte) Zahl x in dieser Gleichung an.
- Analog: Es gibt eine primitiv rekursive Funktion $\delta(n)$, die die (eindeutig bestimmte) Zahl y in der Gleichung $\beta(x, y) = n$ angibt.

Zusammenfassend:

Die Umkehrfunktionen γ und δ der Bijektion β sind primitiv rekursiv. Für alle $n \in \mathbb{N}$ gilt $\beta(\gamma(n), \delta(n)) = n$.

Aus der Bijektion $\beta : \mathbb{N}^2 \rightarrow \mathbb{N}$ kann man weitere Bijektionen zwischen $\mathbb{N}^{k+1}$ und $\mathbb{N}$ bauen: Die **primitiv rekursive** Funktion

$$\beta(n_0, \beta(n_1, \beta(n_2, \dots, \beta(n_{k-1}, n_k) \dots)))$$

weist jedem $(k+1)$ -Tupel $(n_0, n_1, \dots, n_k)$ eine entsprechende Zahl in $\mathbb{N}$ zu, die wir im Folgenden mit $\langle n_0, n_1, \dots, n_k \rangle$ bezeichnen.

Weitere Bijektionen

Aus der Bijektion $\beta : \mathbb{N}^2 \rightarrow \mathbb{N}$ kann man weitere Bijektionen zwischen $\mathbb{N}^{k+1}$ und $\mathbb{N}$ bauen: Die **primitiv rekursive** Funktion

$$\beta(n_0, \beta(n_1, \beta(n_2, \dots, \beta(n_{k-1}, n_k) \dots)))$$

weist jedem $(k+1)$ -Tupel $(n_0, n_1, \dots, n_k)$ eine entsprechende Zahl in $\mathbb{N}$ zu, die wir im Folgenden mit $\langle n_0, n_1, \dots, n_k \rangle$ bezeichnen.

Mit Hilfe der beiden Funktionen $\gamma(n)$ und $\delta(n)$ kann man dann weitere **primitiv rekursive** Funktionen $u_0, u_1, \dots, u_k : \mathbb{N} \rightarrow \mathbb{N}$ konstruieren, sodass für alle $n \in \mathbb{N}$ die folgende Beziehung gilt:

$$n = \langle u_0(n), u_1(n), \dots, u_k(n) \rangle$$

Äquivalenz zu LOOP Programmen

Satz

Die Menge der **primitiv rekursiven** Funktionen fällt mit der Menge der **LOOP-berechenbaren** Funktionen zusammen.

**Hinrichtung:
LOOP → Primitiv rekursiv**

LOOP → Primitiv rekursiv: Hauptidee

- Wir betrachten eine LOOP-berechenbare Funktion f , die von einem LOOP Programm P berechnet wird.
- Die im Programm P verwendeten Variablen seien $x_0, x_1, \dots, x_k$.

LOOP → Primitiv rekursiv: Hauptidee

- Wir betrachten eine LOOP-berechenbare Funktion f , die von einem LOOP Programm P berechnet wird.
- Die im Programm P verwendeten Variablen seien $x_0, x_1, \dots, x_k$.

Wir beweisen mit Induktion über den Aufbau von P , dass eine primitiv rekursive Funktion $g_P : \mathbb{N} \rightarrow \mathbb{N}$ existiert, die die Arbeitsweise von P simuliert:

Wenn das Programm P die Anfangswerte $a_0, a_1, \dots, a_k$ der Variablen in die Endwerte $b_0, b_1, \dots, b_k$ übersetzt, dann gilt entsprechend

$$g_P(\langle a_0, a_1, \dots, a_k \rangle) = \langle b_0, b_1, \dots, b_k \rangle$$

LOOP → Primitiv rekursiv (1)

Zuweisungen

Falls P aus der Zuweisung $x_i := x_j + c$ besteht, so ist

$$g_P(x) = \langle u_0(x), \dots, u_{i-1}(x), u_j(x) + c, u_{i+1}(x), \dots, u_k(x) \rangle$$

LOOP → Primitiv rekursiv (2)

Hintereinanderausführung

Falls P die Form $Q; R$ hat, so gilt

$$g_P(x) = g_R(g_Q(x))$$

LOOP → Primitiv rekursiv (3)

LOOP-Konstrukt

Angenommen, P ist ein LOOP Programm von der Form

LOOP x_i DO Q ENDLOOP

- Wir definieren zunächst die (primitiv rekursive) Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit

$$\begin{aligned}h(0, x) &= x \\h(n+1, x) &= g_Q(h(n, x)).\end{aligned}$$

LOOP → Primitiv rekursiv (3)

LOOP-Konstrukt

Angenommen, P ist ein LOOP Programm von der Form

LOOP x_i DO Q ENDLOOP

- Wir definieren zunächst die (primitiv rekursive) Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit

$$\begin{aligned}h(0, x) &= x \\ h(n+1, x) &= g_Q(h(n, x)).\end{aligned}$$

- Der Aufruf $h(n, x)$ wendet das Programm Q genau n -mal hintereinander auf die Eingabe x an.

LOOP → Primitiv rekursiv (3)

LOOP-Konstrukt

Angenommen, P ist ein LOOP Programm von der Form

LOOP x_i DO Q ENDLOOP

- Wir definieren zunächst die (primitiv rekursive) Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit

$$\begin{aligned}h(0, x) &= x \\h(n+1, x) &= g_Q(h(n, x)).\end{aligned}$$

- Der Aufruf $h(n, x)$ wendet das Programm Q genau n -mal hintereinander auf die Eingabe x an.
- Da der Wert x_i am Anfang der Schleife durch $u_i(x)$ gegeben ist, gilt

$$g_P(x) = h(u_i(x), x)$$

LOOP → Primitiv rekursiv: Schluss

Schlussendlich geben wir Anfang und Ende der Simulation an:

Start des LOOP Programms

Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.

Wenn die m Eingabevariablen die Werte $x'_1, \dots, x'_m$ haben,
so ist der Eingabewert für die primitiv rekursive Simulation

$$x = \langle 0, x'_1, x'_2, \dots, x'_m, \underbrace{0, 0, \dots, 0}_{k-m} \rangle$$

LOOP → Primitiv rekursiv: Schluss

Schlussendlich geben wir Anfang und Ende der Simulation an:

Start des LOOP Programms

Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.

Wenn die m Eingabevariablen die Werte $x'_1, \dots, x'_m$ haben,
so ist der Eingabewert für die primitiv rekursive Simulation

$$x = \langle 0, x'_1, x'_2, \dots, x'_m, \underbrace{0, 0, \dots, 0}_{k-m} \rangle$$

Terminieren des LOOP Programms

Das Resultat ist die Zahl, die sich am Ende in der Variablen x_0 ergibt.

Das Resultat des LOOP Programs P ergibt sich als

$$u_0(g_P(\langle 0, x'_1, x'_2, \dots, x'_m, 0, 0, \dots, 0 \rangle)).$$

**Rückrichtung:
Primitiv rekursiv → LOOP**

Primitiv rekursiv $\rightarrow$ LOOP: Hauptidee

- Wir betrachten eine primitiv rekursive Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$
- Wir beweisen mit Induktion über die Definition von f , dass ein LOOP Programm P_f existiert, das die Auswertung von f simuliert
- Das Programm P_f nimmt die Variablenwerte $x_1, \dots, x_k$ als Eingabe und terminiert mit $x_0 = f(x_1, \dots, x_k)$.

Primitiv rekursiv $\rightarrow$ LOOP (1)

Basisfunktionen

- Die konstante Funktion $f(x_1, \dots, x_k) = c$ wird durch das LOOP Program $x_0 := c$ simuliert.
- Die Projektion $f(x_1, \dots, x_k) = x_j$ wird durch das LOOP Program $x_0 := x_j$ simuliert.
- Die Nachfolgerfunktion $\text{succ}(x_j) = x_j + 1$ wird durch das LOOP Program $x_0 := x_j + 1$ simuliert.

Primitiv rekursiv $\rightarrow$ LOOP (2)

Komposition

Falls die primitiv rekursive Funktion f durch Komposition aus anderen primitiv rekursiven Funktionen $f_1, \dots, f_s$ entsteht, so wird f simuliert, indem man die LOOP Programme für $f_1, \dots, f_s$ geeignet hintereinander ausführt.

Anmerkung: **Zwischenergebnisse** kann sich die Simulation für später merken, indem sie sie in separaten Variablen bis zur Wiederverwendung zwischenspeichert.

Primitiv rekursiv $\rightarrow$ LOOP (3)

Primitive Rekursion

Angenommen, die Funktion $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ entsteht durch primitive Rekursion aus den beiden primitiv rekursiven Funktionen $g : \mathbb{N}^k \rightarrow \mathbb{N}$ und $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$, wobei

$$\begin{aligned}f(0, x_1, \dots, x_k) &= g(x_1, \dots, x_k) \\f(n+1, x_1, \dots, x_k) &= h(n, f(n, x_1, \dots, x_k), x_1, \dots, x_k)\end{aligned}$$

Das simulierende LOOP Programm P_f sieht dann wie folgt aus:

```
x0 := g(x1, ..., xk);  
s := 0;  
LOOP n DO  
    x0 := h(s, x0, x1, ..., xk);  
    s := s + 1;  
ENDLOOP;
```


μ -rekursive Funktionen

Der Kleene'sche μ -Operator

Wir definieren $\min \emptyset = \perp$.

Definition

Es sei $g : \mathbb{N}^{k+1} \rightarrow \mathbb{N} \cup \{\perp\}$ eine (partielle oder totale) Funktion mit $k+1$ Argumenten ($k \geq 1$). Durch Anwendung des μ -**Operators** auf g entsteht eine neue Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N} \cup \{\perp\}$ mit k Argumenten, wobei

$$f(x_1, \dots, x_k) = \min\{n \mid g(n, x_1, \dots, x_k) = 0 \text{ und} \\ \text{für alle } m < n \text{ gilt } g(m, x_1, \dots, x_k) \neq \perp\}.$$

Die entstehende Funktion f wird mit $\mu g : \mathbb{N}^k \rightarrow \mathbb{N} \cup \{\perp\}$ bezeichnet.

Anmerkung: Der μ -Operator arbeitet immer mit der ersten Komponente.

Beispiel

Es sei $g : \mathbb{N}^4 \rightarrow \mathbb{N}$ mit $g(x, a, b, c) = (x - a)^2(x - b)^2(x - c)^2$.

Dann ist $(\mu g)(a, b, c)$ die kleinste Zahl $x \in \mathbb{N}$ mit $g(x, a, b, c) = 0$.

Daraus ergibt sich $(\mu g)(a, b, c) = \min\{a, b, c\}$.

Beispiel

Es sei $g : \mathbb{N}^4 \rightarrow \mathbb{N}$ mit $g(x, a, b, c) = (x - a)^2(x - b)^2(x - c)^2$.

Dann ist $(\mu g)(a, b, c)$ die kleinste Zahl $x \in \mathbb{N}$ mit $g(x, a, b, c) = 0$.

Daraus ergibt sich $(\mu g)(a, b, c) = \min\{a, b, c\}$.

Beispiel

Es sei $g : \mathbb{N}^2 \rightarrow \mathbb{N}$ mit $g(x, y) \equiv 1$.

Dann ist $(\mu g)(y)$ die kleinste Zahl $x \in \mathbb{N}$ mit $g(x, y) = 0$.

Daraus ergibt sich $(\mu g)(y) \equiv \perp$.

Definition

Die Klasse der μ -rekursiven Funktionen ist die kleinste Klasse von (partiellen und totalen) Funktionen,

- die die Basisfunktionen (konstante Funktionen; Projektionen; Nachfolgerfunktion) enthält und
- die abgeschlossen ist unter (1) Komposition, (2) primitiver Rekursion und (3) Anwendung des μ -Operators.

Anmerkung: Wenn eines der Argumente von $f(x_1, \dots, x_k)$ den Wert $\perp$ annimmt, so führt das zum Funktionswert $f(x_1, \dots, x_k) = \perp$.

Stephen Kleene (1909–1994)

Wikipedia: **Stephen Cole Kleene** was a student of Alonzo Church. Kleene is best known as the founder of recursion theory, which subsequently helped to provide the foundations of theoretical computer science.

Kleene's work grounds the study of computable functions. A number of mathematical concepts are named after him: Kleene hierarchy, Kleene algebra, the Kleene star (Kleene closure), Kleene's recursion theorem, and the Kleene fixpoint theorem. He also invented regular expressions, and made significant contributions to the foundations of mathematical intuitionism.



Äquivalenz zu WHILE Programmen

Satz

Die Menge der μ -rekursiven Funktionen fällt mit der Menge der **WHILE-berechenbaren** (Turing-berechenbaren; RAM-berechenbaren) Funktionen zusammen.

- Der Beweis dieses Satz basiert auf dem Beweis unseres letzten Satzes (primitiv rekursiv $\Leftrightarrow$ LOOP-berechenbar), und bildet eine einfache Erweiterung
- In der Hinrichtung zeigen wir, dass WHILE Schleifen mit Hilfe des μ -Operators simuliert werden können
- In der Rückrichtung zeigen wir, dass der μ -Operator durch eine WHILE Schleife simuliert werden kann

Hinrichtung: WHILE $\rightarrow$ μ -rekursiv

Simulation eines WHILE Programms P

WHILE $x_i \neq 0$ DO Q ENDWHILE

- Wir recyceln die Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$, die durch den Aufruf $h(n, x)$ das Programm Q genau n -mal auf x anwendet:

$$h(0, x) = x$$

$$h(n+1, x) = g_Q(h(n, x)).$$

Hinrichtung: WHILE $\rightarrow \mu$ -rekursiv

Simulation eines WHILE Programms P

WHILE $x_i \neq 0$ DO Q ENDWHILE

- Wir recyceln die Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$, die durch den Aufruf $h(n, x)$ das Programm Q genau n -mal auf x anwendet:

$$h(0, x) = x$$

$$h(n+1, x) = g_Q(h(n, x)).$$

- Die Funktion $u_i h : \mathbb{N}^2 \rightarrow \mathbb{N}$ gibt den Wert $u_i h(n, x) = u_i(h(n, x))$ der Variablen x_i nach n -maliger Anwendung von Q an.

Hinrichtung: WHILE $\rightarrow \mu$ -rekursiv

Simulation eines WHILE Programms P

WHILE $x_i \neq 0$ DO Q ENDWHILE

- Wir recyceln die Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$, die durch den Aufruf $h(n, x)$ das Programm Q genau n -mal auf x anwendet:

$$h(0, x) = x$$

$$h(n+1, x) = g_Q(h(n, x)).$$

- Die Funktion $u_i h : \mathbb{N}^2 \rightarrow \mathbb{N}$ gibt den Wert $u_i h(n, x) = u_i(h(n, x))$ der Variablen x_i nach n -maliger Anwendung von Q an.
- Die Funktion $\mu(u_i h) : \mathbb{N} \rightarrow \mathbb{N}$ gibt die kleinste Zahl $n = \mu(u_i h)(x)$ an, für die x_i nach n -maliger Anwendung von Q den Wert 0 hat.

Simulation eines WHILE Programms P

WHILE $x_i \neq 0$ DO Q ENDWHILE

- Wir recyceln die Hilfsfunktion $h : \mathbb{N}^2 \rightarrow \mathbb{N}$, die durch den Aufruf $h(n, x)$ das Programm Q genau n -mal auf x anwendet:

$$h(0, x) = x$$

$$h(n+1, x) = g_Q(h(n, x)).$$

- Die Funktion $u_i h : \mathbb{N}^2 \rightarrow \mathbb{N}$ gibt den Wert $u_i h(n, x) = u_i(h(n, x))$ der Variablen x_i nach n -maliger Anwendung von Q an.
- Die Funktion $\mu(u_i h) : \mathbb{N} \rightarrow \mathbb{N}$ gibt die kleinste Zahl $n = \mu(u_i h)(x)$ an, für die x_i nach n -maliger Anwendung von Q den Wert 0 hat.
- Daher wird das WHILE Programm P simuliert durch

$$g_P(x) = h(\mu(u_i h)(x), x)$$

Simulation des μ -Operators

Angenommen, die Funktion $f : \mathbb{N}^k \rightarrow \mathbb{N}$ entsteht durch den μ -Operator aus der μ -rekursiven Funktion $g : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$, also $f = \mu g$.

Das simulierende WHILE Programm P_f sieht dann wie folgt aus:

```
x0 := 0;  
y := g(0, x1, ..., xk);  
WHILE y ≠ 0 DO  
    x0 := x0 + 1;  
    y := g(x0, x1, ..., xk);  
ENDLOOP;
```

